

Nombre y Apellido: N° Legajo:

Primer Parcial de Programación Imperativa

28/09/2026

- ❖ **Condición mínima de aprobación: Sumar 5 (cinco) puntos.**
- ❖ **Los ejercicios que no se ajusten estrictamente al enunciado, no serán aceptados.**
- ❖ **No usar variables globales ni static.**
- ❖ **No es necesario escribir los #include**
- ❖ **Escribir en esta hoja Nombre, Apellido y Legajo**
- ❖ **Antes de retirarse del aula confirmar con el docente que se haya recibido la entrega**

Ejercicio 1 (3,5 puntos)

Se cuenta con un **vector de enteros no negativos en el rango [0, 15]** almacenado de forma comprimida, donde **cada elemento ocupa 4 bits** (un *nibble*). Por lo tanto, cada byte del vector contiene dos elementos:

Ejemplo: Si el siguiente arreglo contiene 10 elementos:

```
unsigned char arreglo[] = { 0x37, 0xF2, 0x03, 0x4A, 0xFF };
```

Representa al arreglo de los elementos: 3, 7, 15, 2, 0, 3, 4, 10, 15, 15.

Implementar la función `mostRepeated` que recibe los siguientes parámetros

- `v`: vector comprimido
- `dim`: cantidad total de elementos (no de bytes)
- `num`: puntero donde dejar el número que aparece más veces en el vector
- `qty`: puntero donde dejar la cantidad de veces que aparece `num` en el vector `v`

Si no hay elementos en el vector, `num` y `qty` no se modifican

Si hay más de un elemento que aparezca la mayor cantidad de veces, deja cualquiera de ellos en `num`

Programa de test

```
int main(void) {
    unsigned char vec[] = { 0x34, 0xA3, 0xAA };
    int num=1000;
    int qty=900;
    mostRepeated(vec, 0, &num, &qty);
    assert(num==1000);
    assert(qty==900);

    mostRepeated(vec, 1, &num, &qty);
    assert(num==3);
    assert(qty==1);

    mostRepeated(vec, 2, &num, &qty);
    assert(num==3 || num==4);
    assert(qty==1);
}
```

```
mostRepeated(vec, 5, &num, &qty);
assert(num==3 || num==10);
assert(qty==2);

mostRepeated(vec, 6, &num, &qty);
assert(num==10);
assert(qty==3);

puts("OK!");
return 0;
}
```

Ejercicio 2 (3,5 puntos)

Se tienen dos matrices $m1$ y $m2$ de enteros mayores a cero, con la misma cantidad de filas (rows) pero posiblemente distinta cantidad de columnas (cols1 y cols2). Cada fila de $m1$ y de $m2$ está ordenada de menor a mayor, no tiene elementos repetidos y termina con la marca -1. Lo que haya después del -1 no es válido. Una fila puede estar vacía, es decir, contener solo el -1.

Escribir una función que cargue en una tercera matriz $m3$ (de rows filas y cols3 columnas) la unión de $m1$ y $m2$. Cada fila N de $m3$ debe contener los elementos de las filas N de $m1$ y $m2$, ordenados de menor a mayor, sin repetir y terminados en -1.

Si en una fila de $m3$ no entran todos los elementos de la unión junto con el -1, se copian los que quepan y no se coloca el -1. Esto incluye el caso en que los elementos entran justo pero no queda lugar para la marca.

Lo que quede en $m3$ después del -1 no se evalúa.

Prototipo:

```
void unionMat(unsigned int rows,
              unsigned int cols1, int m1[][cols1],
              unsigned int cols2, int m2[][cols2],
              unsigned int cols3, int m3[][cols3]);
```

Programa de prueba

```
// Verifica que dos matrices sean iguales
static void check(unsigned int rows, unsigned int cols3,
                  int got[][cols3], int exp[][cols3]) {
    int ok = 1;
    for (unsigned int r = 0; r < rows; r++) {
        for (unsigned int j = 0; j < cols3; j++) {
            if (got[r][j] != exp[r][j]) return 0;
            if (exp[r][j] == -1) break;
        }
    }
    return ok;
}
```

```
static void test1(void) {
    int m1[3][4] = { {1, 3, 5, -1},
                     {-1},
                     {-1} };
    int m2[3][3] = { {2, 3, -1},
                     {-1},
                     {4, 9, -1} };
    int exp[3][8] = { {1, 2, 3, 5, -1},
                      {-1},
                      {4, 9, -1} };

    int m3[3][8];
    unionMat(3, 4, m1, 3, m2, 8, m3);
    assert(check( 3, 8, m3, exp));
}

static int test2(void) {
    int m1[3][4] = { {1, 3, 5, -1},
                     {1, 2, -1},
                     {-1} };
    int m2[3][4] = { {2, 4, 6, -1},
                     {2, 3, -1},
                     {-1} };
    int exp[3][4] = { {1, 2, 3, 4},
                      {1, 2, 3, -1},
                      {-1} };

    int m3[3][4];
    unionMat(3, 4, m1, 4, m2, 4, m3);
    assert(check( 3, 4, m3, exp));
}

static int test3(void) {
    int m1[2][2] = { {5, -1},
                     {-1} };
    int m2[2][2] = { {3, -1},
                     {-1} };
    int exp[2][1] = { {3},
                      {-1} };

    int m3[2][1];
    unionMat(2, 2, m1, 2, m2, 1, m3);
    assert(check( 2, 1, m3, exp));
}

static int test4(void) {
    int m1[2][4] = { {1, 2, 3, -1},
                     {7, -1} };
    int m2[2][6] = { {1, 2, 3, -1},
                     {1, 7, 9, 11, -1} };
    int exp[2][6] = { {1, 2, 3, -1},
                      {1, 7, 9, 11, -1} };

    int m3[2][6];
    unionMat(2, 4, m1, 6, m2, 6, m3);
    assert(check( 2, 6, m3, exp));
}

int main(void)
```

Nombre y Apellido: N° Legajo:

```
{
    test1();
    test2();
    test3();
    test4();
    puts("Todos los tests pasaron");
    return 0 ;
}
```

Ejercicio 3 (3 puntos)

Escribir una función que reciba dos strings s1 y s2 y elimine de s1 los caracteres que sean iguales al carácter que está en la misma posición de s2. La comparación ignora mayúsculas y minúsculas.

Solo se consideran letras del alfabeto inglés. Un carácter que no sea una de esas letras nunca se elimina, aunque sea idéntico al de la misma posición de s2.

Ejemplos

s1	s2	s1 resultante
"abcdef"	"abXdeY"	"cf"
"Ho1a Mundo"	"h0LA mUNDO"	" "
"a1b2"	"a1c2"	"1b2"
"abcdef"	"abc"	"def"
"aaaa"	"abc"	"aaa"
" "		" "
"123a"	"1a3"	"123a"